3.0.46

Puni Aditya - EE25BTECH11046

March 26, 2026

Question

$P_X(x) = M\exp(-2|x|) + N\exp(-3|x|)$ is the probability density function for the real random variable X, over the entire x axis. M and N are both positive real numbers. The equation relating M and N is

1. $M + \frac{2}{3}N = 1$
2. $2M + \frac{1}{3}N = 1$
3. $M + N = 1$
4. $M + N = 3$

Total Probability Condition

For any valid Probability Density Function (PDF) $P_X(x)$, the total area under the curve over the entire real axis must equal exactly 1:

$$\int_{-\infty}^{\infty} P_X(x)\, dx = 1 \tag{1}$$

Substituting the given PDF:

$$\int_{-\infty}^{\infty} \left(M \exp\left(-2\left|x\right|\right) + N \exp\left(-3\left|x\right|\right)\right)\, dx = 1 \tag{2}$$

Theoretical Solution

Since $\exp(-a|x|)$ is an even function, we evaluate over $[0, \infty)$:

$$\int_{-\infty}^{\infty} \exp(-a|x|)\, dx = 2\int_{0}^{\infty} \exp(-ax)\, dx \tag{3}$$

$$= 2\left[\frac{\exp(-ax)}{-a}\right]_0^{\infty} \tag{4}$$

$$= 2\left(0 - \frac{1}{-a}\right) = \frac{2}{a} \tag{5}$$

Applying this result for $a = 2$ and $a = 3$:

$$M\left(\frac{2}{2}\right) + N\left(\frac{2}{3}\right) = 1 \tag{6}$$

$$M + \frac{2}{3}N = 1 \tag{7}$$

Conclusion: Option (1) is correct.

Python Implementation: Computation

```python
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt

x, t = sp.symbols('x t', real=True)
M, N = sp.symbols('M N', real=True, positive=True)
pdf_sym = sp.Piecewise(
    (M * sp.exp(2 * x) + N * sp.exp(3 * x), x < 0),
    (M * sp.exp(-2 * x) + N * sp.exp(-3 * x), True)
)
total_prob = sp.integrate(pdf_sym, (x, -sp.oo, sp.oo))
M_sol = sp.solve(total_prob - 1, M)[0]

N_val = 0.6
M_val = M_sol.subs(N, N_val)
pdf_sub = pdf_sym.subs({M: M_val, N: N_val})

cdf_sym = sp.integrate(pdf_sub.subs(x, t), (t, -sp.oo, x))
pdf_func = sp.lambdify(x, pdf_sub, modules=['numpy'])
cdf_func = sp.lambdify(x, cdf_sym, modules=['numpy'])

x_vals = np.linspace(-4, 4, 1000)
pdf_vals = pdf_func(x_vals)
cdf_vals = cdf_func(x_vals)
```

Python Implementation: Plotting

```python
plt.figure(1, figsize=(8, 6))
plt.plot(x_vals, pdf_vals, label=rf'PDF $P_X(x)$', color='blue')
plt.axhline(0, color='black', linewidth=2)
plt.axvline(0, color='black', linewidth=2)
plt.fill_between(x_vals, pdf_vals, color='blue', alpha=0.2)
plt.title('Probability Density Function')
plt.xlabel(r'$x$')
plt.ylabel('Probability Density')
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend()
plt.savefig('../figs/pdf_plot.jpg')

plt.figure(2, figsize=(8, 6))
plt.plot(x_vals, cdf_vals, label=r'CDF $F_X(x)$', color='red')
plt.axhline(0, color='black', linewidth=2)
plt.axhline(1, color='black', linewidth=2)
plt.axvline(0, color='black', linewidth=2)
plt.title('Cumulative Distribution Function')
plt.xlabel(r'$x$')
plt.ylabel('Cumulative Probability')
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend()
plt.savefig('../figs/cdf_plot.jpg')
```

Probability Density Function Plot

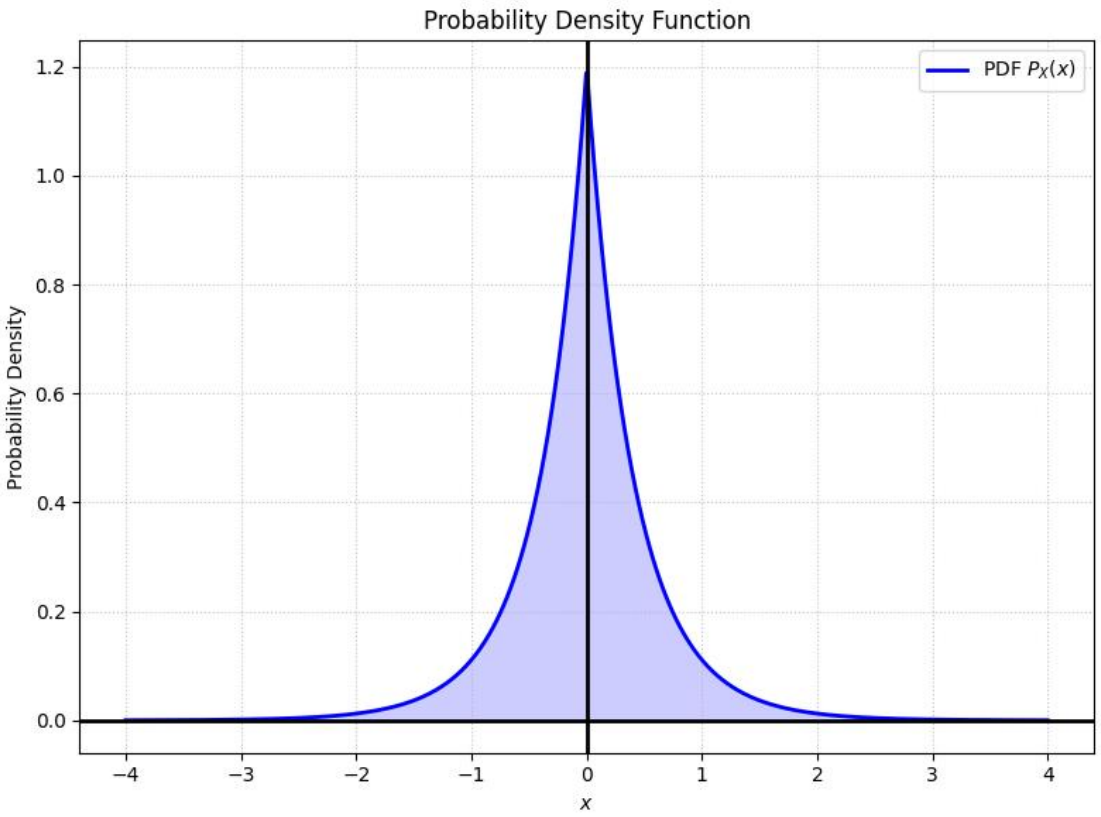

Figure: Plot of the PDF with total shaded area equal to 1

Cumulative Density Function Plot

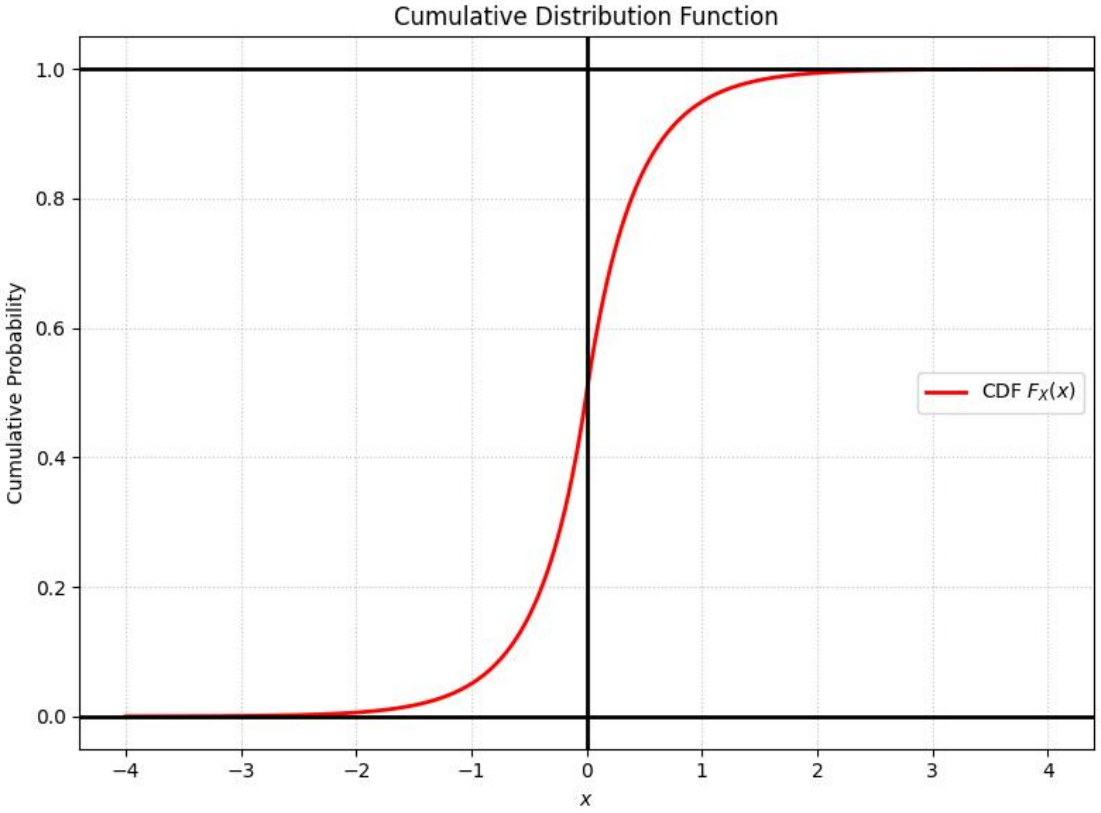

Figure: Plot of the CDF

Generating Laplace via Conditional Gaussian

To generate samples for our PDF, we use the property that a Laplace distribution can be expressed as a **Gaussian Scale Mixture** (a Conditional Gaussian).
If a random variable X follows a Laplace distribution $f(x) = \frac{\lambda}{2} \exp(-\lambda|x|)$, it can be generated in two steps:

1. Draw a variance V from an Exponential distribution with rate parameter $\theta = \frac{\lambda^2}{2}$.
2. Draw X from a Normal (Gaussian) distribution conditioned on that variance V, so $X|V \sim \mathcal{N}(0, V)$.

This powerful property allows us to use standard Gaussian generators to build sharp-peaked, heavy-tailed distributions.

Decomposing the Target PDF

Our target PDF is $P_X(x) = M\exp(-2|x|) + N\exp(-3|x|)$. We must rewrite this as a mixture of standard Laplace PDFs:

$$P_X(x) = M\left(\frac{2}{2}\exp(-2|x|)\right) + N\left(\frac{2}{3}\cdot\frac{3}{2}\exp(-3|x|)\right) \tag{8}$$

$$P_X(x) = \underbrace{M}_{\text{Weight } w_1}\underbrace{\left(\frac{2}{2}\exp(-2|x|)\right)}_{\text{Laplace}(\lambda=2)} + \underbrace{\left(\frac{2N}{3}\right)}_{\text{Weight } w_2}\underbrace{\left(\frac{3}{2}\exp(-3|x|)\right)}_{\text{Laplace}(\lambda=3)} \tag{9}$$

Since $M + \frac{2N}{3} = 1$, this perfectly represents a weighted coin flip between two Laplace distributions.

Algorithm for Sample Generation

Based on the decomposition, we generate samples X as follows:

- **Step 1 (Coin Flip):** Generate $U \sim \text{Uniform}(0, 1)$.
- **Step 2 (Draw Variance V):**
  - If $U < M$: We select the $\lambda = 2$ term. Draw $V \sim \text{Exp}\left(\text{rate} = \frac{2^2}{2} = 2\right)$.
  - If $U \geq M$: We select the $\lambda = 3$ term. Draw $V \sim \text{Exp}\left(\text{rate} = \frac{3^2}{2} = 4.5\right)$.
- **Step 3 (Conditional Gaussian):** Draw our final sample $X \sim \mathcal{N}(0, V)$.

Note: In Python's numpy, the exponential distribution takes the 'scale' parameter, which is $\frac{1}{rate}$.

Python Implementation: Sampling Logic

```python
import numpy as np
import matplotlib.pyplot as plt

N = 0.6
M = 1 - (2/3) * N  # Equals 0.6
num_samples = 10000

u = np.random.rand(num_samples)
is_term = u < M

# For lambda = 2, rate = 2 -> scale = 1/2
# For lambda = 3, rate = 4.5 -> scale = 1/4.5
V = np.zeros(num_samples)
V[is_term] = np.random.exponential(scale=1/2, size=np.sum(is_term))
V[~is_term] = np.random.exponential(scale=1/4.5, size=np.sum(~is_term))

samples = np.random.normal(loc=0, scale=np.sqrt(V))
```

Python Implementation: Plotting

```
plt.figure(figsize=(8, 6))
plt.hist(samples, bins=150, density=True, alpha=0.6, color='dodgerblue',
         edgecolor='black', label='Sampled PDF')

x_vals = np.linspace(-4, 4, 1000)
pdf_theoretical = M * np.exp(-2 * np.abs(x_vals)) + N * np.exp(-3 * np.abs(x_vals))
plt.plot(x_vals, pdf_theoretical, color='red', linewidth=2, linestyle='--',
         label=r'Theoretical: $P_X(x)$')
plt.title('PDF Generation using Conditional Gaussian')
plt.axhline(0, color='black', linewidth=2)
plt.axvline(0, color='black', linewidth=2)
plt.xlabel('x')
plt.ylabel('Probability Density')
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend()
plt.tight_layout()
plt.savefig('../figs/conditional_gaussian.png')
plt.show()
```

Simulation Results (n = 1000)

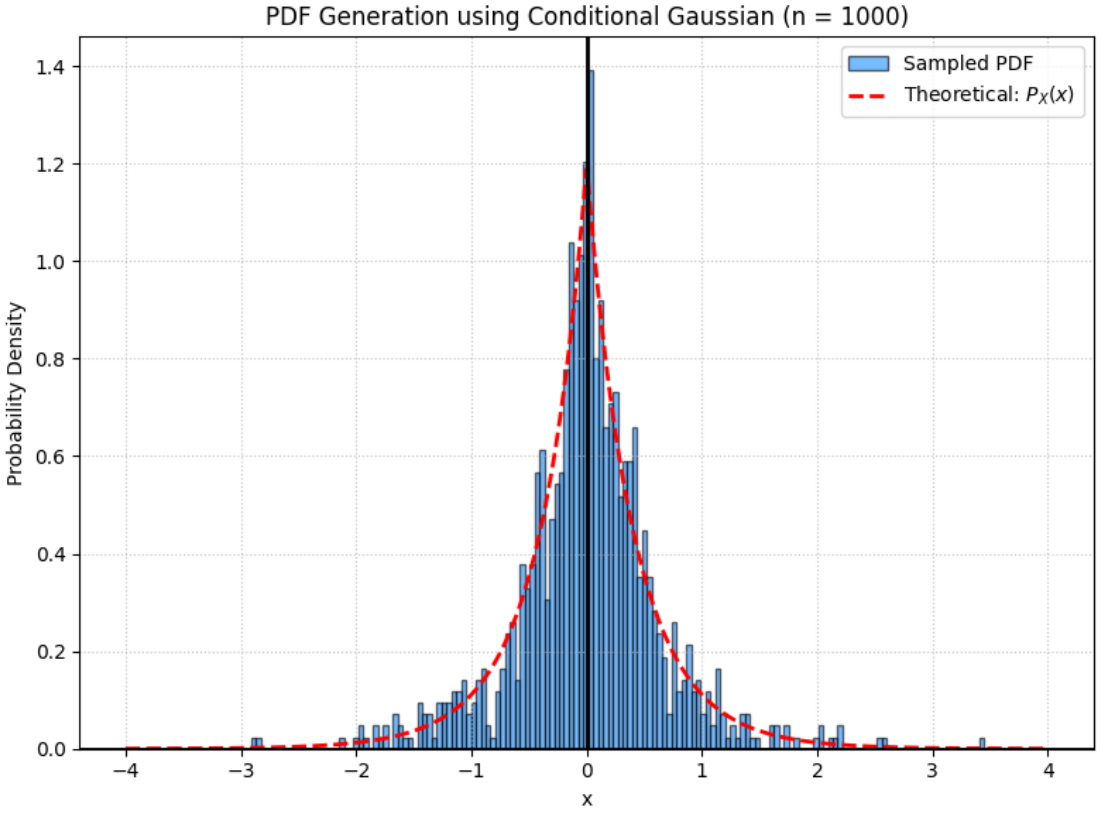

Figure: Empirical PDF generated via Conditional Gaussians vs. Theoretical PDF

Simulation Results (n = 5000)

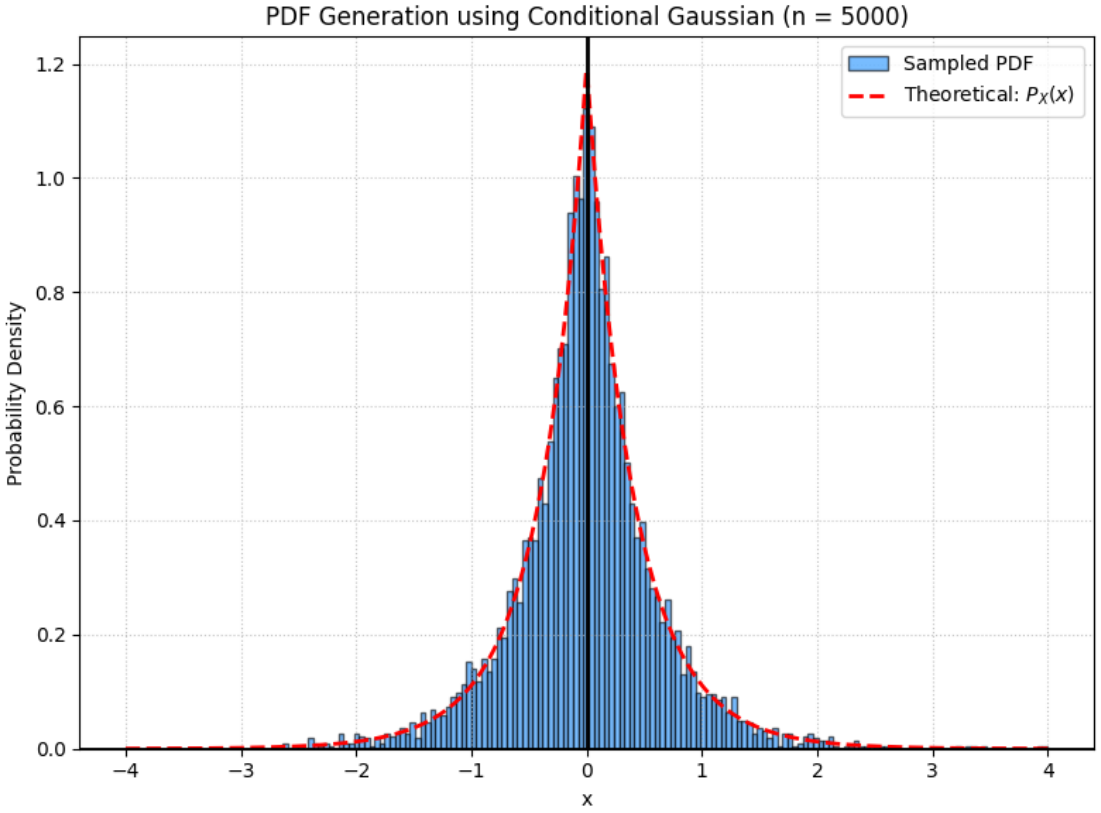

Figure: Empirical PDF generated via Conditional Gaussians vs. Theoretical PDF

Simulation Results (n = 10000)

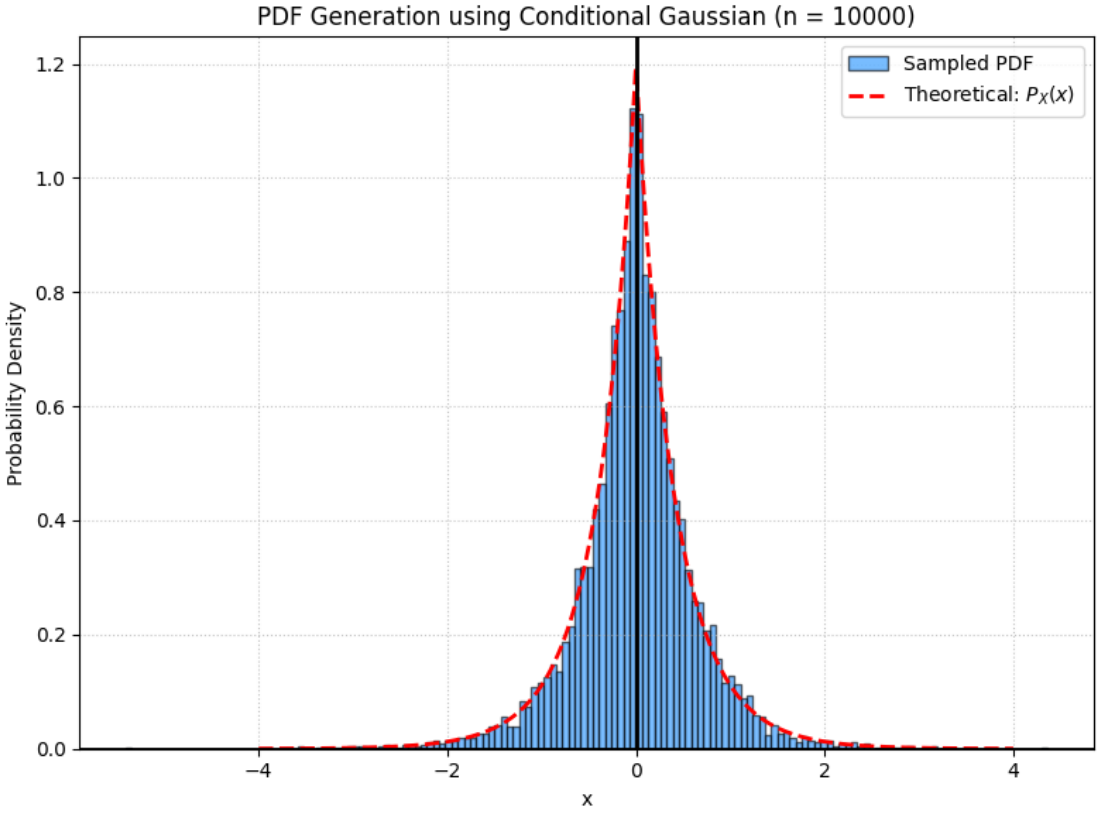

Figure: Empirical PDF generated via Conditional Gaussians vs. Theoretical PDF

Simulation Results (n = 20000)

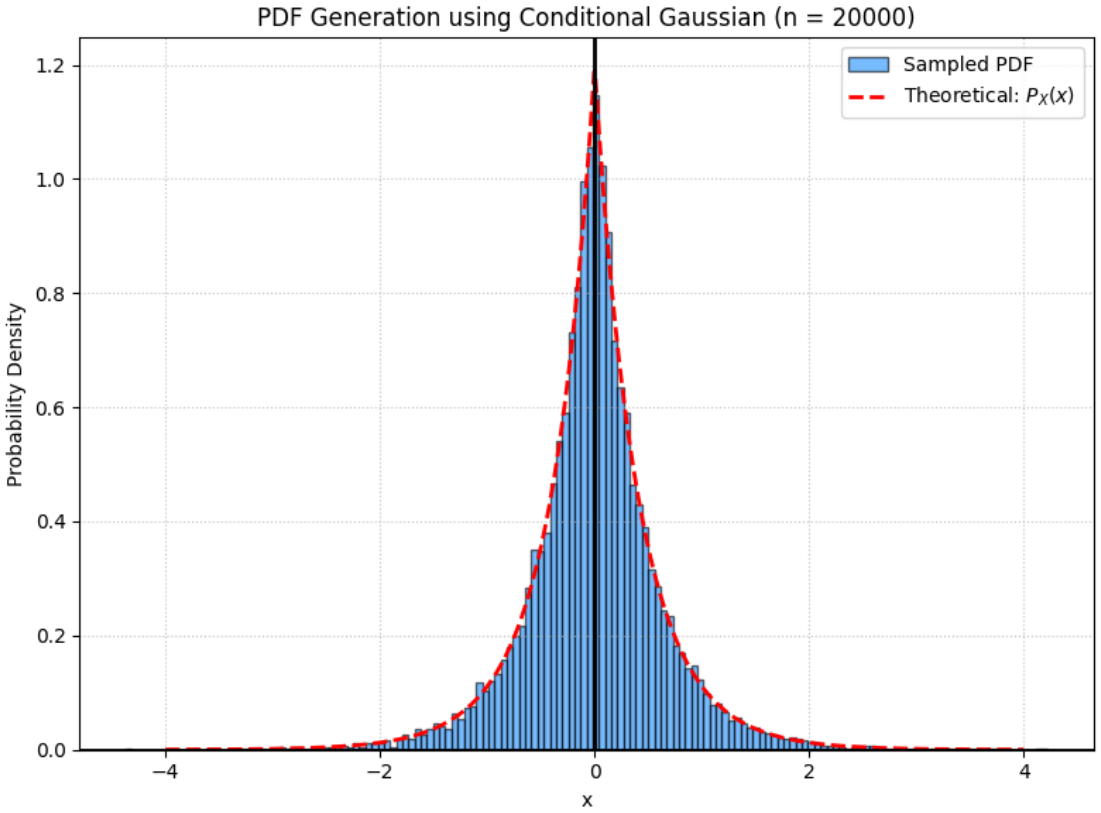

Figure: Empirical PDF generated via Conditional Gaussians vs. Theoretical PDF

The histogram perfectly follows the theoretical curve, verifying our mathematical decomposition and sampling logic.